

INTERNAL ENGINEERING DOCUMENT

POINT OF SALE SYSTEM

MCQ Café POS

Technical Handover & Developer Guide

Architecture

Front-End

Data & Storage

Testing

Deployment

Prepared for the IT / Engineering team · Vietnamese Street Kitchen POS
Code version 40 · 11 June 2026 · ~7,300 lines, zero runtime dependencies

CONFIDENTIAL — FOR INTERNAL HANDOVER USE

Table of Contents

1	Executive Summary & System Overview	
2	Architecture at a Glance	
3	Technology Stack & Design Methodology	
4	Repository & File Structure	
5	The Data Layer & "Database"	store.js · localStorage · IndexedDB · schema · migrations
6	State Management & the Rendering Model	
7	Core Domain Logic	cart · totals & tax · payments · refunds · promotions · queue numbers
8	Front-End Screens & Components	
9	The Self-Service Kiosk (state machine & i18n)	
10	The Media Subsystem (ads, login, customer display)	
11	Security & Access Control	
12	Testing & Quality Assurance	
13	Performance Engineering	
14	Build, Deployment & Operations	
15	Extending the System (recipes)	
16	Known Limitations & Production Roadmap	
A	Appendix — Store API, Storage Schema & Glossary	

1 • Executive Summary & System Overview

MCQ Café POS is a complete, two-faced point-of-sale system — a cashier register and a self-service kiosk — that runs entirely inside a web browser with **no server-side application logic and no build step**. It is written in framework-free ("vanilla") HTML, CSS and JavaScript.

The product serves a single Vietnamese street-food café. One menu, one price book and one sales history are shared between two front-ends:

- **The Register** — staff till: ordering, customisation, split payments, receipts, refunds/voids, reporting, menu & settings administration.
- **The Standing Order Tree** — a customer-facing self-order kiosk: attract loop, bilingual (EN/VI) menu, upsell, loyalty and queue tickets. Kiosk sales drop into the same history the cashier reads.

At a glance

Property	Value
Languages	HTML5, CSS3, ES2017+ JavaScript
Frameworks / build tools	None. No React/Vue, no bundler, no transpiler, no npm runtime dependency
Runtime dependencies	Google Fonts (CDN, optional) — everything else is local
Persistence	<code>localStorage</code> (business data) + <code>IndexedDB</code> (uploaded media blobs)
Source size	~7,300 lines across 4 JS files, 1 stylesheet, 1 HTML shell
Back-end	None required. Optional static-file host (e.g. PythonAnywhere/Flask) only serves the files
Browsers	Chrome / Edge (recommended), Safari, Firefox — desktop & touch

WHY "NO BACKEND"?

The app keeps all state on the device. That makes it trivial to host (any static server or even `file://`), resilient offline, and free of database/operations overhead — at the cost of being single-device. Chapter 16 describes the path to a multi-terminal, server-backed deployment when the business needs it.

2 • Architecture at a Glance

The system is a thin layered single-page application. Four scripts load in dependency order inside one HTML shell; each owns one concern and exposes a single global object.

SHELL • INDEX.HTML

index.html

#view • #modal-root • #print-area • #toast Loads the 4 scripts with a `?v=N` cache-buster.

▲ renders into / reads

PRESENTATION • UI.JS → WINDOW.UI

Views

login • register • payment • history • dashboard •
menu mgr • settings • admin • kiosk • customer
display

Render engine

render() • renderDynamic() • surgical refresh •
modals • toasts • i18n

▲ subscribe(commit) ▼ calls

DOMAIN & STATE • STORE.JS → WINDOW.STORE

State + logic

cart • orders • totals • payments • refunds •
promotions • members • users • audit

Persistence

load() • commit() • persist() → localStorage •
migrations

▼ reads seed ▼ stores blobs

DATA • DATA.JS → WINDOW.MCQ_DATA • MEDIA.JS → WINDOW.MEDIA

Seed dataset

categories • ~70 menu items • modifier groups •
promotions • members • default config

Media store

IndexedDB "mcq-media" • ad / login / customer
slots • object-URL cache

Load order & globals

Scripts load synchronously, bottom of `<body>`, in strict dependency order. Each is an IIFE that attaches exactly one global:

index.html

```
<script src="assets/js/data.js?v=40"></script> // window.MCQ_DATA
<script src="assets/js/store.js?v=40"></script> // window.Store (reads MCQ_DATA)
<script src="assets/js/media.js?v=40"></script> // window.Media (IndexedDB)
<script src="assets/js/ui.js?v=40"></script> // window.UI (reads Store + Media)
```

Global	File	Responsibility
MCQ_DATA	data.js	Immutable seed dataset (menu, categories, promotions, members, default config)
Store / S	store.js	The "model": all business state, logic and persistence. Framework-agnostic.
Media / M	media.js	IndexedDB-backed store for admin-uploaded images/videos + an object-URL cache
UI	ui.js	All rendering and event handlers. Every <code>onClick</code> in markup calls <code>UI.*</code> .

ONE-WAY DEPENDENCIES

`data.js → store.js → ui.js`, plus `media.js → ui.js`. The model (Store) never imports the view (UI). This keeps the domain logic testable in isolation (Chapter 12).

3 • Technology Stack & Design Methodology

The guiding principle is **radical simplicity and longevity**: a system any developer can read end-to-end, that has no dependency tree to rot, and that will still run unchanged in a browser ten years from now.

Why framework-free?

- **Zero build / zero install.** Edit a file, refresh the browser. No node_modules, no webpack, no version drift.
- **No supply-chain surface.** Nothing to `npm audit`; no transitive packages to patch.
- **Full control of performance.** Rendering is explicit, so hot paths (cart, kiosk) update surgically without a virtual-DOM diff (Chapter 13).
- **Longevity.** The platform *is* the standard library — DOM, `localStorage`, `IndexedDB`, `<video>`.

Patterns & methods used

Pattern	Where & why
Module pattern (IIFE)	Every file is <code>(function(global){ ... })(window)</code> — private scope, one explicit export. Replaces ES modules without a bundler.
Single source of truth	One <code>state</code> object in <code>store.js</code> holds everything; the UI is a pure function of it.
Observer / pub-sub	<code>Store.subscribe(fn)</code> + <code>commit()</code> . Any mutation calls <code>commit()</code> , which persists and notifies the UI to re-render.
Declarative views, imperative patches	Views are template-string render functions; high-frequency updates patch only the changed sub-tree.
Finite state machine	The kiosk (<code>attract → home → menu → member → pay → done</code>) and the payment flow are explicit stages.
Schema versioning & migration	<code>seedVersion</code> on the config drives forward-migration of persisted data (Chapter 5).
Graceful degradation	Persistence, media storage and autoplay all wrapped in <code>try/catch</code> and feature checks so a blocked API never breaks the app.

Domain modelling method

The POS domain is modelled as plain data with pure transforms. An **order** is a list of **lines**; a line references a menu item plus chosen add-ons, modifiers, note and quantity. Pricing is a pure function `computeTotals(cart)` — no side effects, fully unit-testable. Payment, refund and promotion rules are likewise expressed as data-in/data-out functions over that model.

CONVENTION

Money is integer-safe via `round2()`; identifiers are short slugs (`banh-mi`, `u-admin`); time buckets use `dayKey(ts)` for day-grouped reports.

4 • Repository & File Structure

The repository is intentionally flat. Source is under `assets/`; documentation under `docs/`.

repository tree

```
Pos-mcq-system/
├─ index.html           // app shell – loads the 4 scripts (~21 lines)
├─ assets/
│  └─ js/
│     ├── data.js       // seed dataset → MCQ_DATA (~278 lines)
│     ├── store.js      // model: state, logic, storage (~1,153 lines)
│     ├── media.js      // IndexedDB media store (~129 lines)
│     └─ ui.js          // all views + handlers (~4,013 lines)
│  └─ css/
│     └─ styles.css     // full theme + responsive + print (~1,718 lines)
│  └─ images/           // logo, backgrounds, per-item photos
│  └─ videos/           // ad1/ad2 (HD attract) + secondary clips
└─ docs/
   ├── USER_GUIDE.html // end-user manual (source)
   ├── MCQ-Cafe-POS-User-Guide.pdf
   ├── TECH_GUIDE.html // this document (source)
   ├── img/             // guide screenshots
   └─ README.md
```

File	Owns	Key exports / globals
<code>data.js</code>	Catalogue & defaults	<code>MCQ_DATA</code> .{ <code>CATEGORIES</code> , <code>MENU</code> , <code>MODIFIER_GROUPS</code> , <code>PROMOTIONS</code> , <code>MEMBERS</code> , <code>CONFIG</code> }
<code>store.js</code>	Business model	<code>Store.*</code> (see Appendix A) — ~90 functions
<code>media.js</code>	Uploaded media	<code>Media</code> .{ <code>preload</code> , <code>add</code> , <code>remove</code> , <code>clearSlot</code> , <code>ads</code> , <code>login</code> , <code>customer</code> }
<code>ui.js</code>	Everything visual	<code>UI.*</code> — render functions + DOM event handlers
<code>styles.css</code>	Look & feel	CSS custom properties, responsive & <code>@media print</code> rules

5 • The Data Layer & "Database"

There is no SQL database and no server. The persistence tier is two browser storage APIs wrapped by `store.js` and `media.js`.

5.1 Business data — localStorage

The entire live business state is serialised to a single `localStorage` key. Reads/writes are wrapped so a blocked storage API (private mode) degrades to in-memory only.

`store.js`

```
const KEY = 'mcq_pos_state_v1';

function persist(){ try{ localStorage.setItem(KEY, JSON.stringify(state)); }catch(e){} }
function commit(){ persist(); emit(); } // mutate → save → notify UI
function subscribe(fn){ listeners.add(fn); return () ⇒ listeners.delete(fn); }
```

The persisted object (the "schema") is a single JSON document:

Key	Shape	Notes
<code>config</code>	store details, tax, logo, users, flags, <code>seedVersion</code>	Editable in Settings; also holds media flags (Chapter 10)
<code>menu</code>	array of items (price, availability, category, image)	Seeded from <code>MCQ_DATA.MENU</code> , then user-editable
<code>promotions</code>	date-aware discount rules	Auto activate/expire by date
<code>cart</code>	the in-progress order (lines, type, discounts)	The live register order
<code>orders</code>	completed sales (newest first)	Shared by counter + kiosk; queried for receipts & refunds
<code>held</code>	parked tickets	Recall/void
<code>auditLog</code>	sensitive-event trail	Sales, refunds, voids, discounts, settings changes
<code>credits</code> / <code>tills</code>	goodwill credits, till readings	Cash management
<code>live</code>	customer-display channel	Mirrors the active cart to the second screen

5.2 Schema migration

On `load()`, a stored `seedVersion` below the current (`MCQ_DATA.CONFIG.seedVersion`, presently **12**) triggers forward-migration — adopting new default fields without clobbering user edits. This is how the codebase ships breaking dataset changes safely to devices that already hold data.

`store.js` • `migrateConfig()`


```
const savedVer = savedConfig.seedVersion || 1;
if (savedVer ≥ (D.CONFIG.seedVersion || 1)) return; // already current
// ...adopt new defaults only where the user never customised...
state.config.seedVersion = D.CONFIG.seedVersion;
```

5.3 Uploaded media — IndexedDB

Images/videos uploaded by an admin are far too large for `localStorage` (~5 MB cap), so `media.js` stores them as **Blobs in IndexedDB**. Synchronous render code can't await IndexedDB, so the module keeps an in-memory cache of `Object URLs` built once at boot by `preload()`.

`media.js` • object store

```
DB "mcq-media" → object store "media" { keyPath:"id", index:"slot" }
record = { id, slot, name, type, kind, size, ord, blob, createdAt }
slots = "ad" | "login" | "customer" // each a rotating list

cache = { ad:[...urls], login:[...urls], customer:[...urls] } // rebuilt on every add/remove
```

WHY OBJECT URLS

Blobs are turned into `blob:` URLs once and reused, so a `<video src>` streams straight from memory. Old URLs are `revokeObjectURL`'d on each rebuild to avoid leaks. See Chapter 10 for the full subsystem.

6 • State Management & the Rendering Model

The UI is a function of state. A mutation calls `Store.commit()`; the store persists and emits; the UI re-renders. The art is re-rendering *only* what changed.

The render pipeline

ui.js

```
function init(){
  Store.load();
  // build #view / #modal-root / #print-area / #toast shells
  Store.subscribe(renderDynamic); // re-render on every commit
  render();
  if (Media) Media.preload().then(repaintMediaViews); // swap in uploaded media when ready
}

function renderDynamic(){
  // SURGICAL fast-paths - avoid blowing away the whole view:
  if (ui.view === 'register' && cartEl) return refreshCart(); // only #pos-cart
  if (ui.view === 'backscreen') return paintBackScreen(); // only the order half
  if (ui.view === 'kiosk') return; // kiosk drives its own paints
  renderTopbar(); render(); // otherwise full repaint
}
```

Three tiers of update

Tier	Mechanism	Used by
Full	<code>render()</code> rewrites <code>#view.innerHTML</code> from a template string	Navigation between views
Surgical	Replace only one sub-element (e.g. <code>#pos-cart</code> , <code>.cust2-order</code> , a kiosk card)	High-frequency state (cart edits, kiosk +/-)
Out-of-band	Modals in <code>#modal-root</code> ; receipts in <code>#print-area</code> ; toasts	Dialogs, printing, transient messages

For example, adding an item at the register never rebuilds the 70-item menu grid — only the cart panel is re-rendered, preserving its scroll position. On the kiosk, the stepper updates a single card and the bottom bar in place. This is what keeps the touch UI flicker-free (Chapter 13).

EVENT WIRING

There is no event-delegation framework: markup uses inline `onClick="UI.fn(args)"`. Every handler is therefore an explicit member of the `UI` export object, which doubles as the public surface for tests.

7 • Core Domain Logic

All pricing and fulfilment rules live in `store.js` as pure-ish functions over the data model. This chapter maps the most important ones.

Cart & line construction

`buildLine(item, opts)` composes a cart line from a menu item plus chosen add-ons, modifiers, note and quantity. `addItem` / `setQty` / `incQty` / `updateLine` / `removeLine` mutate the cart and `commit()`.

Totals & tax

`computeTotals(cart)` is the single pricing authority: it sums lines, applies per-line and order-level discounts, resolves active promotions, computes tax and (optional) cash rounding, and returns a totals object. Being pure, it is the most heavily unit-tested function in the system.

Payments

- **Split tender** across Cash / Card / Other, each partial amount tracked until the balance is cleared.
- **Cash** keypad with automatic change calculation and optional rounding.
- **Card (EFTPOS)** is a *timed simulation* — no real processor. Replacing this is the #1 production task (Chapter 16).

Completing, refunding & voiding

`store.js` — order lifecycle

```
completeOrder(payment) // cart → orders.unshift(order); assigns daily queue no.; audits; clears cart
findOrder(id) · searchOrders(q) // receipt lookup by id / queue / amount / name
refundOrder(id, lines) · voidOrder(id)
refundableQty(...) · remainingRefundable(...) // partial-refund guards
```

Promotions

Promotions are **date-aware**: `promotionActiveNow()` / `promotionStatus()` switch rules on and off automatically by date, so seasonal offers need no manual toggling.

Shared daily queue number

KEY INTEGRATION DETAIL

The counter and the kiosk draw from the **same** daily sequence. A kiosk sale is `orders.unshift`'d into the very same history list, so the cashier can look it up, reprint its receipt, or refund it — there is no separate "kiosk orders" silo.

Loyalty

`getMembers` / `findMember` / `addMemberPoints` back a simple points scheme (seed members `MCQ1001-1003`), surfaced at both the till and the kiosk.

8 • Front-End Screens & Components

Every screen is a render function in `ui.js` returning a template string, selected by `render()` on `ui.view`.

View	Render fn	Purpose
Login	<code>renderLogin</code>	Account picker → PIN pad; rotating background (Chapter 10)
Register	<code>renderPOS</code>	Category grid, search, cart, customise, discounts, hold
Payment	payment modal flow	Split tender, cash keypad, card simulation
History	<code>renderOrders</code>	Search sales, reprint, refund, void
Dashboard	<code>renderDashboard</code>	KPIs, category mix, payment mix, PDF/Excel export
Menu Manager	<code>renderMenuManager</code>	Prices, availability, add-on groups, promotions
Settings	<code>renderSettings</code>	Store details, logo, staff, tax, backup/restore
Admin Hub	<code>renderAdminHub</code>	Launcher: back-office tools + the three media managers
Customer Display	<code>renderBackScreen</code>	Guest-facing live order over rotating media
Kiosk	<code>renderKiosk</code>	The self-service Standing Order Tree (Chapter 9)

Cross-cutting UI services

- **Modals** — `openModal/closeModal/modalShell/confirmModal` render into `#modal-root`.
- **PIN pad** — surgical: only the four dots update per keypress; the keypad never rebuilds.
- **Printing** — receipts/reports are composed into `#print-area` and produced via `window.print()` + `@media print` CSS (no PDF library).
- **Toasts** — `toast(msg)` for transient confirmations.

9 • The Self-Service Kiosk

The Standing Order Tree is a full-screen state machine with its own cart held in UI state (not the store), an idle auto-reset, and bilingual copy.

Stages

ui.js — kiosk FSM

```
attract → home → menu (+cart) → member → pay → done(queue ticket)
ui.kioskStage drives renderKiosk(); kioskPaint() rewrites #view directly
armKioskIdle() // inactivity → wipe cart, return to attract (abandoned-order safety)
```

Internationalisation (EN / VI)

A tiny i18n layer — `kt(key)` over a `KT_DICT` dictionary — localises kiosk copy; a header toggle flips `ui.kioskLang` and repaints.

Surgical kiosk updates

Each menu card carries a `data-id` and an always-present quantity badge. A tap routes through `kioskSyncCard(id)` + `kioskSyncBar()` which update just that card and the checkout bar — so **no image ever reloads** and the grid never rebuilds.

Attract advertising & neon frame

The attract screen plays the ad playlist (Chapter 10) inside a CSS-only glowing neon frame with hanging lanterns and corner lights. The frame animates `opacity` only, so it composites cheaply.

10 • The Media Subsystem

Three admin "media managers" let staff run their own photos/videos on the kiosk ad loop, the login screen and the customer display — with no back-end. This is the newest subsystem; IT should understand it well.

Storage model (media.js)

One IndexedDB object store, three logical **slots**, each an ordered list. Uploading appends; the in-memory **cache** of object URLs is rebuilt after every change so synchronous render code stays simple.

media.js — public API

```
Media.preload()           // boot: blobs → object-URL cache
Media.add(slot, file)      // store a Blob, rebuild cache
Media.remove(id) · clearSlot(slot)
Media.ads() · login() · customer() // synchronous getters → [{id,name,kind,size,url}]
```

Built-in + uploaded, the same logic on all three screens

Each manager presents one unified list:

- **Built-in defaults** (the clips shipped in **assets/videos/**) appear as real, *removable* rows. "Removing" a built-in records its id in a config flag — it is not deleted from disk.
- **Uploads** are appended and stored in IndexedDB.
- When a screen holds more than one item, items **rotate** — images on a timer, videos on **onended**.
- A **Restore** action clears the hidden-defaults flag.

ui.js — the "active list" pattern (ads shown)

```
const DEFAULT_ADS = [{ id:'def-ad1', url:'assets/videos/ad1.mp4?v=2', builtin:true }, ... ];

function activeAds(){
  const hidden = Store.getConfig().kioskAdsHiddenDefaults || [];
  return DEFAULT_ADS.filter(d => !hidden.includes(d.id))
    .concat(Media.ads()); // built-ins (kept) + uploads
}
```

Screen	Slot	Built-in	Config flag(s)
Kiosk attract	ad	ad1.mp4, ad2.mp4 (HD)	kioskAdsHiddenDefaults , kioskAdsOff
Login screen	login	3-panel collage	loginHiddenDefault , loginVideoOff
Customer display	customer	vid-847.mp4	customerHiddenDefault

OPERATIONAL CONSEQUENCE

Media lives in the browser of the device that uploaded it. There is no sync across machines — each terminal manages its own content. A future CMS would centralise this (Chapter 16).

11 • Security & Access Control

Access control is role- and PIN-based, enforced in the client. It is appropriate for an in-store, single-tenant till — but it is **not** a server-grade security boundary, and the document is explicit about that.

Roles & gating

Account	PIN (default)	Role
Manager	1234	admin
Cashier 1	1111	cashier
Cashier 2	2222	cashier

- `GATED_VIEWS` (admin, dashboard, menu, settings, audit) require a manager; `render()` bounces a cashier back to the register.
- `requireAdmin(cb)` unlocks admin features once per shift on a manager PIN; `requireAdminAlways(cb)` re-prompts every time (e.g. discounts).
- Transaction History is open to all; only the *refund action* inside it is gated.

THREAT MODEL — BE HONEST WITH STAKEHOLDERS

PINs and data live in the browser. This protects against casual misuse by staff, not a determined attacker with device access (data is readable in DevTools / `localStorage`). Card payments are simulated. For handling real money and PII at scale, move auth and payments server-side (Chapter 16). **Change the default PINs before going live.**

12 • Testing & Quality Assurance

Because the model is decoupled from the DOM, it is tested two ways: static checks, and headless-browser behavioural smoke tests driving the real `UI` surface.

Static & syntax

Every change is validated with `node --check` on each JS file (parse-level guarantee) before it is run in a browser.

Headless behavioural tests (puppeteer-core + system Chrome)

A real Chrome loads the app from a local static server and exercises features through the public `window.UI` / `window.Media` / `window.Store` globals — the same surface the markup calls. Assertions check DOM outcomes and that the console is error-free.

example smoke test (abridged)

```
await page.evaluate(() => { UI.chooseCashier('u-admin'); ['1','2','3','4'].forEach(UI.pinKey); });
await page.evaluate(async () => { await Media.add('ad', file); await Media.preload(); });
// assert: ad rows rendered, blob URLs, rotation advances, zero console errors
```

Representative coverage already exercised: media add/remove/restore across all three slots, rotation, login PIN entry & the video-pause performance fix, kiosk surgical add/remove, and the customer-display rotation.

Mocked-DOM unit tests (Node `vm`)

Domain functions (`computeTotals` , order lifecycle) can be loaded into a Node `vm` with a minimal DOM mock for fast, browser-free assertions.

RECOMMENDED NEXT STEP

Commit the smoke tests into a `test/` folder and wire a `package.json` script + GitHub Action so they run on every push. The harness already exists; it just needs to live in the repo.

13 • Performance Engineering

On touch hardware, perceived smoothness is everything. The codebase applies a consistent set of techniques to keep frames cheap.

Technique	Where
Surgical re-render — patch the cart / one card, never the grid	Register cart, kiosk steppers
Stable media elements — keep a playing <code><video></code> alive across data updates	Customer display order repaint
Pause background video under modals — a frosted (backdrop-blur) overlay over a moving video re-blurs every frame; pausing freezes the layer so the compositor caches it	Fixes login PIN-entry lag — all modals
GPU-friendly animation — animate <code>transform</code> / <code>opacity</code> , not layout; <code>will-change</code> hints; honour <code>prefers-reduced-motion</code>	Neon frame, attract marquee, CTA
Media weight discipline — HD only where it sells (the attract ads); secondary clips re-encoded H.264, audio stripped, <code>+faststart</code> so they stream instantly; <code>poster</code> + <code>preload</code>	assets/videos
Cache-busting — <code>?v=N</code> query bumped on release so clients fetch fresh CSS/JS	index.html

ffmpeg recipe used for secondary clips

```
ffmpeg -i in.mp4 -an \
-vf "scale='if(gte(iw,ih),min(1280,iw),-2)':'if(gte(iw,ih),-2,min(1280,ih))'" \
-c:v libx264 -preset slow -crf 26 -pix_fmt yuv420p -movflags +faststart out.mp4
```

14 • Build, Deployment & Operations

There is nothing to build. "Deploying" means serving the static files; "releasing" means bumping the cache-buster.

Local development

```
# any static server from the repo root
python3 -m http.server 8080      # → http://localhost:8080
```

Edit a file, hard-refresh the browser. No watch process, no compile.

Hosting

- Any static host works: PythonAnywhere (Flask serving the folder + a static-files mapping), Netlify, Cloudflare Pages, GitHub Pages, S3+CloudFront, or a plain nginx.
- For media-heavy smoothness, serve `/assets/` through a CDN / static mapping rather than the app process.

Release procedure

1. Make and test changes (`node --check` + headless smoke).
2. Bump `?v=N` on the CSS/JS tags in `index.html` (current: **40**).
3. Commit, push, redeploy the static files, reload the host.
4. Hard-refresh open devices once.

Backup & restore

Settings → Export/Import writes/reads the whole `localStorage` document as JSON. This is the operational backup path per device. Uploaded media (IndexedDB) is not included in that JSON and must be re-uploaded if a device is wiped.

15 • Extending the System

Common change recipes, with the exact touch-points.

Add a menu item

Add to `MCQ_DATA.MENU` in `data.js` (id, name, price, category, image) and bump `seedVersion` if existing devices must adopt it — or add it at runtime via Menu Manager. Item photos live in `assets/images/items/<id>.jpg`.

Add a new screen/view

1. Write `renderXYZ()` returning a template string.
2. Add a branch in `render()` on `ui.view === 'xyz'`.
3. Expose any handlers on the `UI` export object; reference them as `onClick="UI.fn()"`.
4. If it is manager-only, add it to `GATED_VIEWS`.

Add a new media slot

1. In `media.js`: add the slot to `cache` and the `preload` / `revokeAll` branches; add a getter.
2. In `ui.js`: define defaults + an `active<Slot>()` list, a manager modal mirroring the existing three, and a rotation arm if needed.
3. Add a tile in `renderAdminHub` and export the handlers.

Add a business rule (e.g. a tax band, a promotion type)

Implement it as a pure function in `store.js` and route it through `computeTotals`; add a unit test. Keep all pricing in one place.

HOUSE STYLE

Match the surrounding code: 2-space indent, single quotes, terse helpers, comments that explain *why*. New markup uses inline `UI.*` handlers; new state changes end in `commit()`.

16 • Known Limitations & Production Roadmap

The architecture is deliberately single-device and client-only. These are the conscious trade-offs and the upgrade path when the business outgrows them.

Current limitations

Limitation	Impact
Data is per-device (localStorage)	No shared history across terminals; backup is manual JSON export
Media is per-device (IndexedDB)	Each terminal must be configured separately
Card payments are simulated	No real settlement
Client-side auth	Not a hard security boundary; data visible in the browser
No real-time multi-terminal sync	Two tills can't share one live order book

Recommended path to production

1. **Back-end API + database** (e.g. Node/Python + Postgres) as the source of truth for menu, orders, members and audit. The current **Store** becomes a thin client over an HTTP/realtime API — its function surface barely changes.
2. **Real payments** via a PSP (Stripe/Adyen/Square) and/or a hardware terminal, replacing the EFTPOS simulation.
3. **Server-side auth** (hashed PINs/credentials, sessions, per-action authorisation) replacing client gating.
4. **Central media CMS** — move the three media managers' content to the server so head office pushes ads/branding to all kiosks, with scheduling (dayparting).
5. **Multi-terminal sync** (websockets) for a shared live order book and instant menu/price propagation.
6. **CI** — repo-committed tests on every push (the harness already exists).

MIGRATION IS INCREMENTAL

Because the model/view split is clean, each step above can be adopted on its own. The fastest first win is putting **orders** behind an API so multiple terminals share one sales history.

A • Reference

A.1 — Store public API (window.Store)

Group	Functions
Lifecycle	<code>load</code> • <code>subscribe</code> • <code>commit</code>
Getters	<code>getConfig</code> • <code>getMenu</code> • <code>getCart</code> • <code>getHeld</code> • <code>getOrders</code> • <code>getAuditLog</code> • <code>getCategories</code> • <code>getPromotions</code> • <code>findItem</code> • <code>itemsByCategory</code>
Pricing	<code>computeTotals</code> • <code>linePromo</code> • <code>money</code> • <code>round2</code>
Cart	<code>addItem</code> • <code>setQty</code> • <code>incQty</code> • <code>removeLine</code> • <code>updateLine</code> • <code>buildLine</code> • <code>setOrderType</code> • <code>setTable</code> • <code>setCustomer</code> • <code>setPager</code> • <code>setCartNote</code> • <code>setDiscount</code> • <code>setLineDiscount</code> • <code>clearAllDiscounts</code> • <code>clearCart</code>
Auth / staff	<code>getUsers</code> • <code>verifyUser</code> • <code>verifyAdminPin</code> • <code>setActiveCashier</code> • <code>addUser</code> • <code>updateUser</code> • <code>deleteUser</code>
Orders	<code>completeOrder</code> • <code>voidOrder</code> • <code>refundOrder</code> • <code>findOrder</code> • <code>lastOrder</code> • <code>searchOrders</code> • <code>refundableQty</code> • <code>remainingRefundable</code>
Held / credits / till	<code>holdOrder</code> • <code>recallHeld</code> • <code>deleteHeld</code> • <code>getCredits</code> • <code>addCredit</code> • <code>tillReport</code> • <code>ringOff</code> • <code>getTills</code>
Modifiers / menu	<code>getModifierGroups</code> • <code>groupsForItem</code> • <code>addonsUnit</code> • <code>displayName</code> • <code>upsertMenuItem</code> • <code>deleteMenuItem</code> • <code>toggleAvailability</code>
Promotions	<code>togglePromotion</code> • <code>addPromotion</code> • <code>updatePromotion</code> • <code>deletePromotion</code> • <code>promotionStatus</code> • <code>promotionActiveNow</code>
Loyalty	<code>getMembers</code> • <code>findMember</code> • <code>addMemberPoints</code>
Customer display	<code>getLive</code> • <code>setLive</code> • <code>clearLive</code>
Reports / data	<code>report</code> • <code>dayKey</code> • <code>updateConfig</code> • <code>resetAll</code> • <code>exportData</code> • <code>importData</code>

A.2 — Storage schema

Store	Location	Key / structure
Business data	localStorage	<code>mcq_pos_state_v1</code> → JSON { config, menu, promotions, cart, orders, held, auditLog, credits, tills, live }
Uploaded media	IndexedDB	DB <code>mcq-media</code> → store <code>media</code> (keyPath <code>id</code> , index <code>slot</code>)
Media flags	localStorage (in config)	<code>kioskAdsHiddenDefaults</code> • <code>kioskAdsOff</code> • <code>loginHiddenDefault</code> • <code>loginVideoOff</code> • <code>customerHiddenDefault</code>

A.3 — Glossary

Term	Meaning
Standing Order Tree	The self-service kiosk front-end
Attract loop	The full-screen ad video shown before a customer interacts
Slot	A media channel (ad / login / customer), each a rotating list
Built-in default	A clip shipped with the app; listed and removable, hidden via a config flag rather than deleted
Surgical render	Updating one element instead of re-rendering the whole view
commit()	persist state to localStorage and notify the UI to re-render
seedVersion	Dataset schema version driving forward-migration of persisted data

DOCUMENT SCOPE

This handover reflects code version 40. Pair it with the end-user *MCQ Café POS — User Guide* for operational procedures. The source is the ground truth; this document maps it.